

AI ENGINEERING INTERN · POC ASSIGNMENT

Document Intelligence Pipeline.

Build a working MVP that ingests a PDF, extracts structured content with Claude, publishes a governed knowledge page, and verifies its own accuracy. A scaled-down version of an architecture pattern we run in production — distilled to a single happy-path workflow.

CANDIDATE

Nidhi Bharti

IIT Kharagpur

EFFORT

~25 hrs

self-paced

DURATION

7–10 days

part-time

BUDGET

\$0 AUD

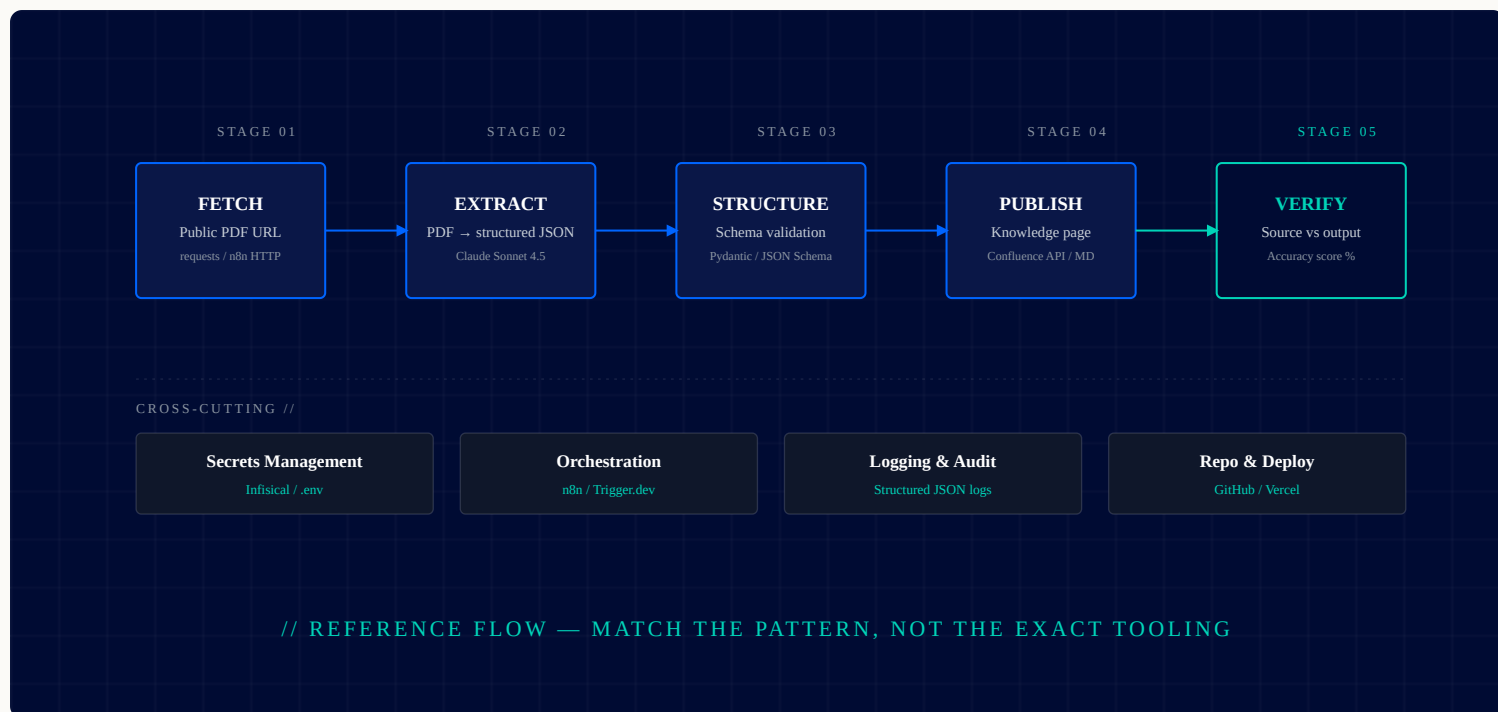
free tier only



A NOTE FROM THOMPSON

Nidhi — your RAG and LangChain background gives you a head start here. This isn't a tutorial assignment — it's a scaled-down version of a production pipeline pattern we run for an enterprise engagement. We want to see how you make architectural decisions, manage secrets properly, and handle the boring-but-critical parts (verification, error handling, docs). One end-to-end thing that works beats five half-built things.

Build a **Document Intelligence Pipeline MVP** that fetches a public PDF, extracts and structures its content using the Anthropic API, publishes the result as a governed knowledge page, and runs a verification pass that scores the extraction against the source. End-to-end, working, on free tiers, demoable.



03 // The Stack

LAYER • LLM

REQUIRED

Anthropic API

Claude Sonnet 4.5 (or Haiku for cheaper iterations). Use the official SDK. Hit the messages endpoint directly — no LangChain wrapper needed for this scale. Free credit will be provided.

LAYER • SECRETS

REQUIRED

Infisical

Don't commit API keys to GitHub. Sign up for the free Infisical tier, store your secrets there, and pull them at runtime via their CLI or SDK. This is non-negotiable — we audit every repo.

LAYER • REPO

REQUIRED

GitHub (Public)

Single public repo. Clean README with setup instructions, architecture diagram (Mermaid or image), and a demo Loom link. Use conventional commits if you can.

n8n OR Trigger.dev

Your choice. n8n.cloud free tier (or self-hosted via Docker) for visual workflow. Trigger.dev for code-first scheduled jobs. Pick the one that fits your pipeline pattern best — justify it in the README.

Confluence OR Markdown

Stretch goal: publish extracted page to Atlassian Cloud Free (10-user tier). Easier path: render as Markdown to a folder in the repo. Both are acceptable for MVP.

Vercel / Railway

Optional: if you wrap the pipeline in a tiny FastAPI or Next.js trigger UI, deploy it to a free tier. Not required for grading — but nice to have for the demo.

MVP Scope

IN SCOPE

What you build.

- + One end-to-end happy path workflow
- + One sample PDF (suggested: any public regulation, policy, or technical standard, ~10–40 pages)
- + Claude API call with a structured-output prompt
- + JSON schema validation of extracted output
- + Verification stage — re-prompt with source + output, get accuracy %
- + Output destination (Markdown file OR Confluence page)
- + README, architecture diagram, Loom walkthrough (~5 min)

What to skip.

- ✗ Multi-tenant or user auth — single-user is fine
- ✗ UI polish — a CLI or basic page is enough
- ✗ Multi-PDF batch processing
- ✗ Production error handling, retry logic, dead letter queues
- ✗ Cost optimisation or token-usage telemetry
- ✗ OCR for scanned PDFs — pick a text-native PDF
- ✗ Fine-tuning, embeddings, or vector storage

05 //

Deliverables

01

Public GitHub Repository

Clean, commented code with a README that includes architecture diagram, setup steps, environment variables, and a known-issues section.

MANDATORY

02

Working Pipeline Run

I should be able to clone, set env vars, run one command, and see a PDF processed end-to-end. Include the sample PDF in the repo or link it.

MANDATORY

03

Verification Output

A JSON or Markdown file showing the verification stage result for at least one document. Aim for >85% accuracy score using your own rubric.

MANDATORY

04

5-Minute Loom Walkthrough

Walk me through your architecture, the trickiest decision you made, and a live run. No slides — screen-share the code and terminal.

MANDATORY

Architecture Decision Record (ADR)

A single Markdown file in `/docs/ADR-001.md` explaining your choice of orchestrator (n8n vs Trigger.dev), prompt design, and what you'd build next with more time.

[STRETCH](#)

How We Grade

01

Does it actually work?

End-to-end. I will clone and run it. If `README` setup is broken, that's a flag.

02

Prompt engineering quality

Is the extraction prompt thoughtful, structured, and robust? Did you use system prompts, XML tags, examples?

03

Secret hygiene

No keys in repo. Infisical or environment variable boundaries respected. `.gitignore` sane.

04

Architectural reasoning

Can you explain why you chose what you chose? The Loom and ADR matter as much as the code.

05

Verification self-honesty

If your accuracy was 60%, say so and explain why. Inflated metrics are worse than honest low ones.

06

Documentation discipline

README quality, code comments, commit messages. The boring parts that show production maturity.

10 days. Self-paced.

DAY 1–2

Scaffold & Read

Repo, Infisical, sample PDF. Read MCP & Anthropic docs.

DAY 3–5

Build the Spine

Fetch → Extract → Structure. Get the happy path working in a script.

DAY 6–8

Publish & Verify

Add publish target. Build the verification stage. Tune accuracy.

DAY 9–10

Polish & Ship

README, diagram, Loom, ADR. Pull request closure check.

// SUBMISSION PROTOCOL

When you're done, send one email.

- 01 To: **thompson@di.net.au** · CC: yourself
- 02 Subject: **[POC Submission] Nidhi Bharti — Document Intelligence Pipeline**
- 03 Body: GitHub URL, Loom URL, any deployed app URL, and a short paragraph on what you'd change with more time
- 04 Make your GitHub **data analytics repository** public before you send (per our intro call)
- 05 I'll review within 48 hours and book a follow-up call to walk through the work

Thompson Cherian

Head of Pre-Sales · Design Industries

T 1300 73 63 63 · WWW.DI.NET.AU